

Architectural Justification

We chose Microservices Architecture for the Alumni Mentorship & Mock Interview Platform, modelled around UC-04: Book Mentorship Session.

Architectural Choice

The system is decomposed into independently deployable services: a Student Portal (UI) Component, Mentor Matching Service, Availability Service, Booking Service, Notification Service, and a Session Database Component. The Booking Service acts as the orchestrator for the booking flow, coordinating with Availability and Notification through defined interfaces.

Reason 1: Uneven, Peak-Driven Load

NFR-002 requires supporting at least 500 concurrent booking transactions with 99.5% monthly uptime and sub-2-second response times, with load spiking during placement season. The Booking and Availability services will see disproportionately higher traffic than Notification or the Matching engine, and microservices allow these hot-path services to be scaled independently without over-provisioning the entire system.

Reason 2: Fault Isolation Matches the Use Case's Own Failure Handling

UC-04's alternate flows already assume partial failure is normal: a slot can be taken by a race condition (5a), or session creation can fail and must roll back cleanly (6a). A microservices boundary between Booking and its dependencies (Availability, Notification, Database) ensures a downtime or slowdown in a downstream service (e.g., Notification) cannot corrupt or block the Booking Service's own transaction and rollback logic.

Security Advantage

NFR-001 requires profile and scorecard data to follow privacy guidelines and support soft-deletion within 24 hours. Splitting services means each microservice only holds and exposes the data it owns (e.g., only the Booking Service and Database Component can write session records), narrowing the blast radius of any single compromised service and simplifying the scope of access control and audit logging for sensitive data.

Performance Benefit

During placement-season traffic spikes, the Booking and Availability services can be horizontally scaled on their own, keeping the average response time under the 2-second NFR-002 target without needing to scale lower-traffic services like Notification or the future Moderation service, which would waste resources under a monolithic layered design.